Day 13 — Database Fundamentals

Student Management Database Design

Objective: Design a simple relational database for managing students, courses, and student enrollments. The design uses three related tables: students, courses, and enrollments.

1. Introduction to Databases

A database is an organized collection of information that can be stored, searched, updated, and managed efficiently. A relational database stores information in tables that are connected through keys.

2. Tables, Rows, and Columns

A table stores information about one type of item. A row is one record in a table, and a column stores one attribute or field. For example, the students table stores student records, and each row represents one student.

3. Primary Keys and Foreign Keys

Primary key (PK): A field that uniquely identifies each row in a table. It cannot be NULL and must be unique.
Foreign key (FK): A field that refers to a key in another table and helps maintain valid relationships between records.

4. Database Design

Database name: student_management

Table 1: students

Stores the basic details of each student.

Field	Data type	Constraint / purpose
student_id	SERIAL	Primary key
full_name	VARCHAR(100)	Student's full name; required
email	VARCHAR(150)	Required and unique
date_of_birth	DATE	Date of birth; optional

Table 2: courses

Stores the details of each course.

Field	Data type	Constraint / purpose
course_id	SERIAL	Primary key
course_name	VARCHAR(100)	Required and unique
credits	INTEGER	Must be greater than 0

Table 3: enrollments

Connects students with the courses they take. This table handles the many-to-many relationship.

Field	Data type	Constraint / purpose
enrollment_id	SERIAL	Primary key

Field	Data type	Constraint / purpose
student_id	INTEGER	Foreign key → students.student_id
course_id	INTEGER	Foreign key → courses.course_id
enrollment_date	DATE	Date of enrollment; defaults to current date

5. PostgreSQL SQL Commands

Create a database in PostgreSQL. Run the first command in a PostgreSQL query tool connected to the default postgres database; then connect to student_management before running the table commands.

```
CREATE DATABASE student_management;
```

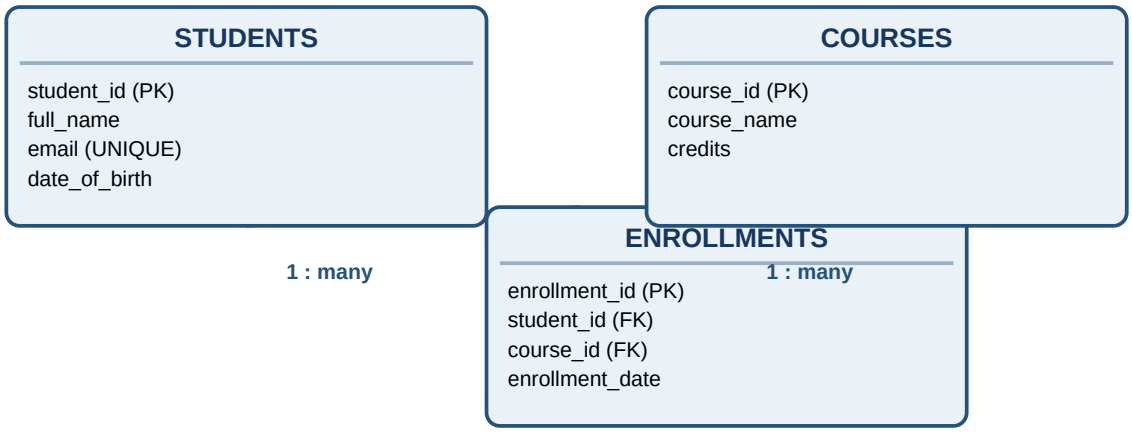
After connecting to student_management, create the tables:

```
CREATE TABLE students ( student_id SERIAL PRIMARY KEY, full_name VARCHAR(100) NOT NULL, email
VARCHAR(150) UNIQUE NOT NULL, date_of_birth DATE ); CREATE TABLE courses ( course_id SERIAL PRIMARY
KEY, course_name VARCHAR(100) UNIQUE NOT NULL, credits INTEGER CHECK (credits > 0) ); CREATE TABLE
enrollments ( enrollment_id SERIAL PRIMARY KEY, student_id INTEGER NOT NULL REFERENCES
students(student_id), course_id INTEGER NOT NULL REFERENCES courses(course_id), enrollment_date DATE
DEFAULT CURRENT_DATE, UNIQUE (student_id, course_id) );
```

The UNIQUE (student_id, course_id) constraint prevents the same student from being enrolled in the same course more than once.

6. Entity Relationship (ER) Diagram

The diagram shows students and courses linked through the enrollments table. One student can have many enrollment records, and one course can appear in many enrollment records.



A student can have multiple enrollments; a course can have multiple enrollments.
The enrollments table connects students and courses (many-to-many relationship).

Relationship summary:

• students.student_id → enrollments.student_id (one-to-many)
• courses.course_id → enrollments.course_id (one-to-many)
• students and courses have a many-to-many relationship through enrollments.

7. Database Design Basics

The design separates student details from course details to reduce repeated information. The enrollments table stores the links between them. Primary keys identify records, while foreign keys keep relationships valid.

8. How to Run and Check

1. Install PostgreSQL or use an available PostgreSQL server.
2. Open pgAdmin or another PostgreSQL query tool.
3. Connect to the server and create the student_management database.
4. Connect to the new database and run the CREATE TABLE statements.
5. Refresh the database tree and confirm that students, courses, and enrollments appear.

Expected output: a Student Management database with three related tables and an ER diagram describing their relationships.